The ways in which healthy modern projects resolve these five issues are discussed in more detail next. Because the resolutions of issues 4 and 5 are tightly coupled, they are discussed together in Section 15.4. Sections 15.5 and 15.6 discuss modern projects in the context of my top 10 software management principles and an alternative set of software best practices.

15.1 CONTINUOUS INTEGRATION

Iterative development produces the architecture first, allowing integration to occur as the verification activity of the design phase and enabling design flaws to be detected and resolved earlier in the life cycle. This approach avoids the big-bang integration at the end of a project by stressing continuous integration throughout the project. Figure 15-1 illustrates the differences between the progress profile of a healthy modern project and that of a typical conventional project, which was introduced in Figure 1-2. The architecture-first approach forces integration into the design phase through the construction of demonstrations. The demonstrations do not eliminate the design breakage; instead, they make it happen in the engineering stage, when it can be resolved efficiently in the context of life-cycle goals. The downstream integration nightmare, late patches, and shoe-horned software fixes are avoided. The result is a more robust and maintainable design.

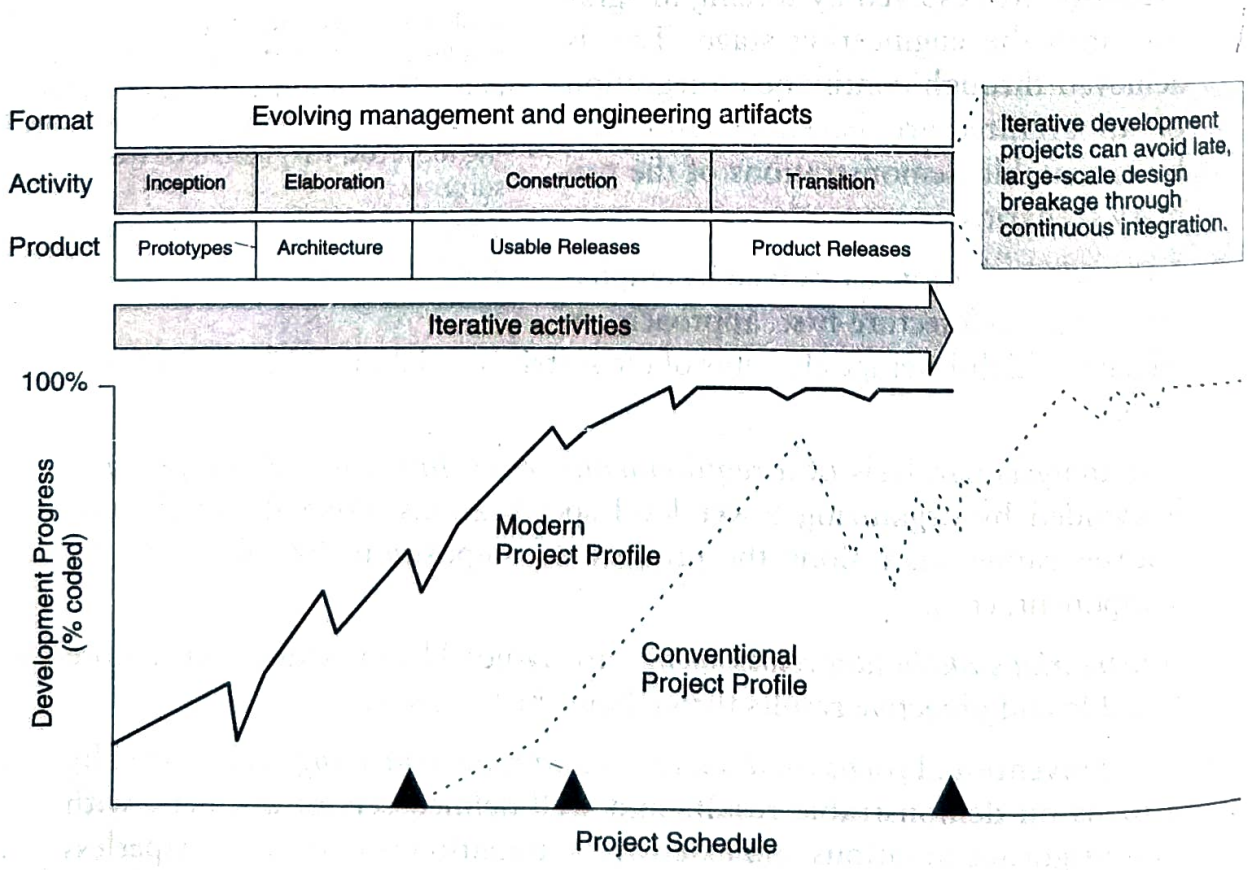


FIGURE 15-1. *Progress profile of a modern project*

TABLE 15-1. *Differences in workflow cost allocations between a conventional process and a modern process*

SOFTWARE ENGINEERING WORKFLOWS	CONVENTIONAL PROCESS EXPENDITURES	MODERN PROCESS EXPENDITURES
Management	5%	10%
Environment	5%	10%
Requirements	5%	10%
Design	10%	15%
Implementation	30%	25%
Assessment	40%	25%
Deployment	5%	5%
Total	100%	100%

The continuous integration inherent in an iterative development process also enables better insight into quality trade-offs. System characteristics that are largely inherent in the architecture (performance, fault tolerance, maintainability) are tangible earlier in the process, when issues are still correctable without jeopardizing target costs and schedules. A recurring theme of successful iterative development projects is a cost profile very different from that experienced by conventional processes.

Table 15-1 identifies the differences in a modern process profile from the perspective of the cost distribution among the various project workflows. This table is a simple combination of Table 1-1 (a typical conventional allocation) and Table 10-1 (a default modern allocation). In my experience, the primary discriminator of a successful modern process is inherent in the overall life-cycle expenditures for assessment and testing. Conventional projects, mired in inefficient integration and late discovery of substantial design issues, expend roughly 40% or more of their total resources in integration and test activities. Modern projects with a mature, iterative process deliver a product with only about 25% of the total budget consumed by these activities.

15.2 EARLY RISK RESOLUTION

The engineering stage of the life cycle (inception and elaboration phases) focuses on confronting the risks and resolving them before the big resource commitments of the production stage. Conventional projects usually do the easy stuff first, thereby demonstrating early progress. A modern process attacks the important 20% of the requirements, use cases, components, and risks. This is the essence of my most important principle: architecture first. Defining the architecture rarely includes simple steps

for which visible progress can be achieved easily. The effect of the overall life-cycle philosophy on the 80/20 lessons learned over the past 30 years of software management experience provides a useful risk management perspective.

- **80% of the engineering is consumed by 20% of the requirements.** Strive to understand the driving requirements completely before committing resources to full-scale development. Do not strive prematurely for high fidelity and full traceability of the requirements.
- **80% of the software cost is consumed by 20% of the components.** Elaborate the cost-critical components first so that planning and control of cost drivers are well understood early in the life cycle.
- **80% of the errors are caused by 20% of the components.** Elaborate the reliability-critical components first so that assessment activities have enough time to achieve the necessary level of maturity.
- **80% of software scrap and rework is caused by 20% of the changes.** Elaborate the change-critical components first so that broad-impact changes occur when the project is nimble.
- **80% of the resource consumption (execution time, disk space, memory) is consumed by 20% of the components.** Elaborate the performance-critical components first so that engineering trade-offs with reliability, changeability, and cost-effectiveness can be resolved as early in the life cycle as possible.
- **80% of the progress is made by 20% of the people.** Make sure that the initial team for planning the project and designing the architecture is of the highest quality. An adequate plan and adequate architecture can then succeed with an average construction team. An inadequate plan or inadequate architecture will probably not succeed, even with an expert construction team.

Figure 15-2 compares the risk management profile of a modern project with the profile for a typical conventional project presented in Figure 1-3.

15.3 EVOLUTIONARY REQUIREMENTS

Conventional approaches decomposed system requirements into subsystem requirements, subsystem requirements into component requirements, and component requirements into unit requirements. The organization of requirements was structured so that traceability was simple. With an early life-cycle emphasis on requirements first, design second, then complete traceability between requirements and design

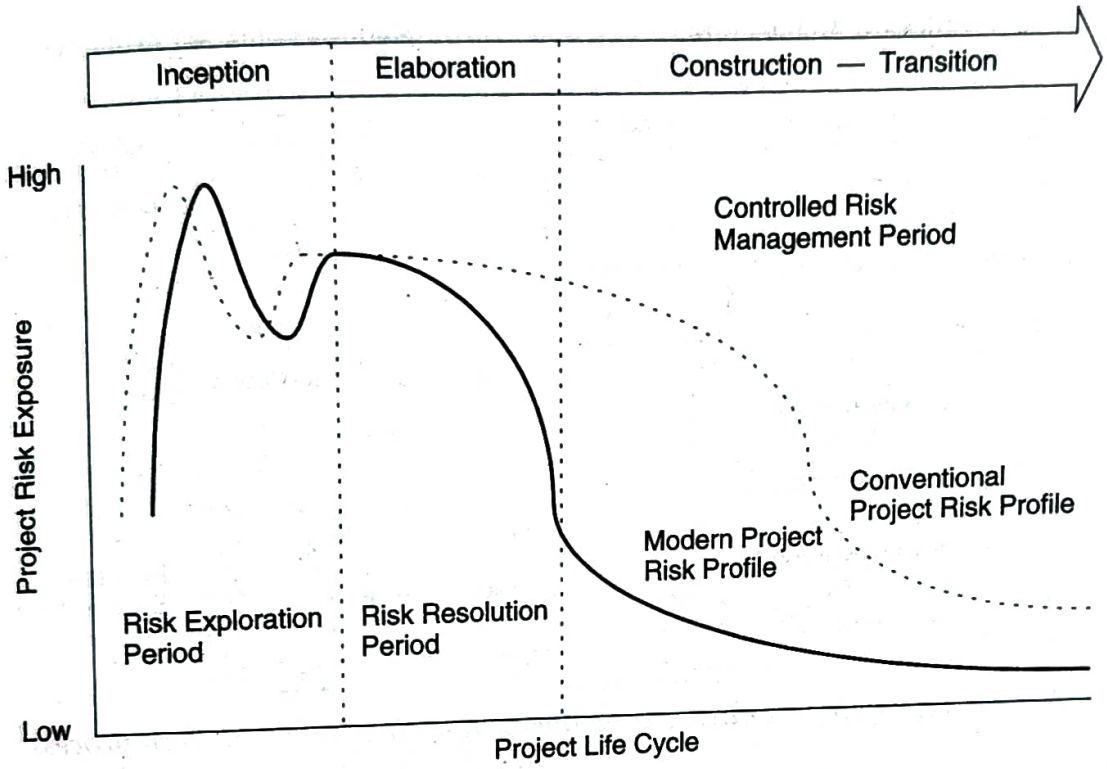


FIGURE 15-2. *Risk profile of a typical modern project across its life cycle*

components, the natural tendency was for the design structure to evolve into an organization that closely paralleled the structure of the requirements organization. It was no surprise that functional decomposition of the problem space led to a functional decomposition of the solution space.

Most modern architectures that use commercial components, legacy components, distributed resources, and object-oriented methods are not trivially traced to the requirements they satisfy. There are now complex relationships between requirements statements and design elements, including 1 to 1, many to 1, 1 to many, conditional, time-based, and state-based.

Top-level system requirements are retained as the vision, but lower level requirements are captured in evaluation criteria attached to each intermediate release. These artifacts, illustrated in Figure 15-3, are intended to evolve along with the process, with more and more fidelity as the life cycle progresses and requirements understanding matures. This is a fundamental difference from conventional requirements management approaches, in which this fidelity was pursued far too early in the life cycle.

15.4 TEAMWORK AMONG STAKEHOLDERS

Many aspects of the classic development process cause stakeholder relationships to degenerate into mutual distrust, making it difficult to balance requirements, product features, and plans. A more iterative process, with more-effective working relationships

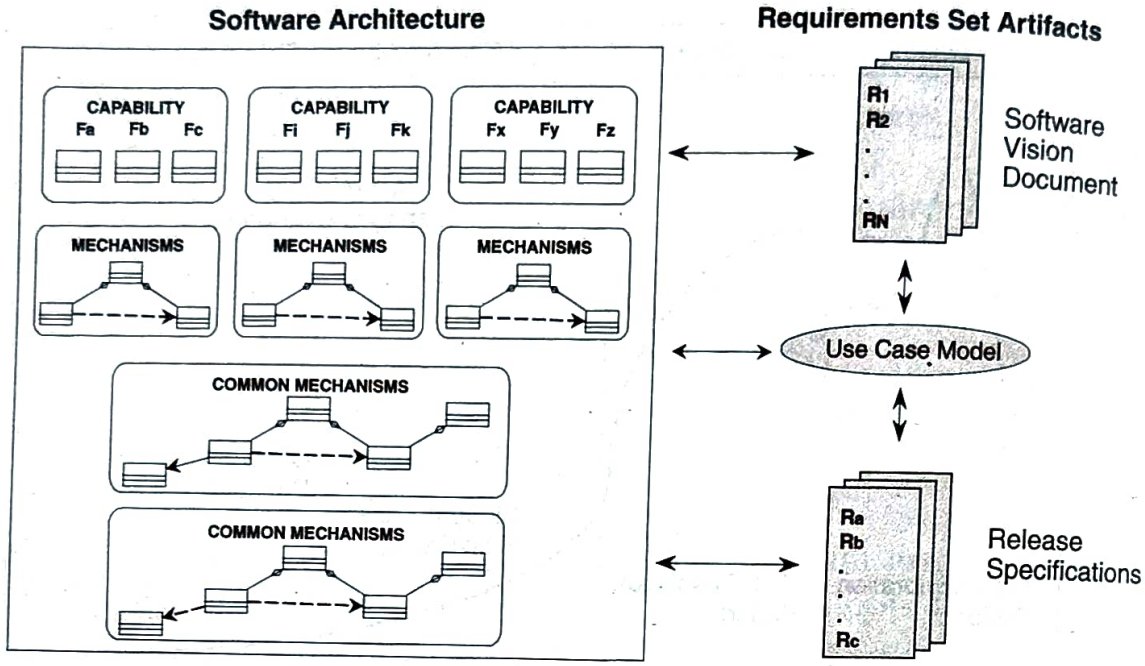


FIGURE 15-3. *Organization of software components resulting from a modern process*

between stakeholders, allows trade-offs to be based on a more objective understanding by everyone. This process requires that customers, users, and monitors have both applications and software expertise, remain focused on the delivery of a usable system (rather than on blindly enforcing standards and contract terms), and be willing to allow the contractor to make a profit with good performance. It also requires a development organization that is focused on achieving customer satisfaction and high product quality in a profitable manner.

The transition from the exchange of mostly paper artifacts to demonstration of intermediate results is one of the crucial mechanisms for promoting teamwork among stakeholders. Major milestones provide tangible results and feedback from a usage point of view. As Table 15-2 shows, designs are now guilty until proven innocent: The project does not move forward until the objectives of the demonstration have been achieved. This prerequisite does not preclude the renegotiation of objectives once the demonstration and major milestone results permit further understanding of the trade-offs inherent in the requirements, design, plans, and technology.

In Table 15-2, the apparent results may still have a negative connotation. A modern iterative process that focuses on demonstrable results (rather than just briefings and paper) requires all stakeholders to be educated in the important distinction between apparently negative results and evidence of real progress. For example, a design flaw discovered early, when the cost to resolve it is tenable, can often be viewed as positive progress rather than as a major issue.

TABLE 15-2. *Results of major milestones in a modern process*

APPARENT RESULT	REAL RESULT
Early demonstrations expose design issues and ambiguities in a tangible form.	Demonstrations expose the important assets and risks of complex software systems early, when they can be resolved within the context of life-cycle goals.
The design is noncompliant (so far).	Understanding of compliance matures from important perspectives (architecturally significant requirements and use cases).
Driving requirements issues are exposed, but detailed requirements traceability is lacking.	Requirements changes are considered in balance with design trade-offs.
The design is considered "guilty until proven innocent."	Engineering progress and issues are tangible, for incorporation into the next iteration's plans.

15.5 TOP 10 SOFTWARE MANAGEMENT PRINCIPLES

My top 10 software management principles were introduced in Chapter 4 as a backdrop to the software process framework and its underlying tenets. To summarize a modern project profile, the following paragraphs revisit each of these principles and describe the project expectation associated with the successful application of each principle. In essence, the list provides a concise, top-level description of the features and benefits of a modern process as viewed by a software project manager.

1. Base the process on an ***architecture-first*** approach. An early focus on the architecture results in a solid foundation for the 20% of the stuff (requirements, components, use cases, risks, errors) that drives the overall success of the project. Getting the architecturally important components to be well understood and stable before worrying about the complete breadth and depth of the artifacts should result in scrap and rework rates that decrease or remain stable over the project life cycle.
2. Establish an ***iterative life-cycle process*** that confronts risk early. A more dynamic planning framework supported by an iterative process results in better risk management and more predictable performance. Resolving the critical issues first results in a predictable construction phase with no surprises, as well as minimal exposure to sources of cost and schedule unpredictability.
3. Transition design methods to emphasize ***component-based development.*** The complexity of a software effort is mostly a function of the number of human-generated artifacts. Making the solution smaller reduces management complexity.

4. Establish a ***change management environment.*** The dynamics of iterative development, including concurrent workflows by different teams working on shared artifacts, necessitate highly controlled baselines.
5. Enhance change freedom through tools that support ***round-trip engineering.*** Automation enables teams to spend more time on engineering and less time on overhead tasks.
6. Capture design artifacts in rigorous, ***model-based notation.*** An engineering notation for design enables complexity control, objective assessment, and automated analyses.
7. Instrument the process for ***objective quality control*** and progress assessment. Progress and quality indicators are derived directly from the evolving artifacts, providing more-meaningful insight into trends and correlation with requirements.
8. Use a ***demonstration-based approach*** to assess intermediate artifacts. Integration occurs early and continuously throughout the life cycle. Intermediate results are objective and tangible.
9. Plan intermediate releases in groups of usage scenarios with ***evolving levels of detail.*** Requirements, designs, and plans evolve in balance. Useful software releases are available early in the life cycle.
10. Establish a ***configurable process*** that is economically scalable. Methods, techniques, tools, and experience can be applied straightforwardly to a broad domain, providing improved return on investment across a line of business.

Throughout this book I have emphasized the importance of balance. From numerous perspectives, the software project manager's paramount objective is to maintain the proper balance of emphasis across the 10 principles. Figure 15-4 summarizes this balance theme in the context of the fundamental software economics equation.

15.6 SOFTWARE MANAGEMENT BEST PRACTICES

Many software management best practices have been captured by various authors and industry organizations. One of the most visible efforts has been the Software Acquisition Best Practices Initiative, sponsored by the U.S. Department of Defense to "improve and restructure our software acquisition management process." Brown summarized the initiative [Brown, 1996], which has three components: the Airlie Software Council (composed of software industry gurus), seven different issue panels

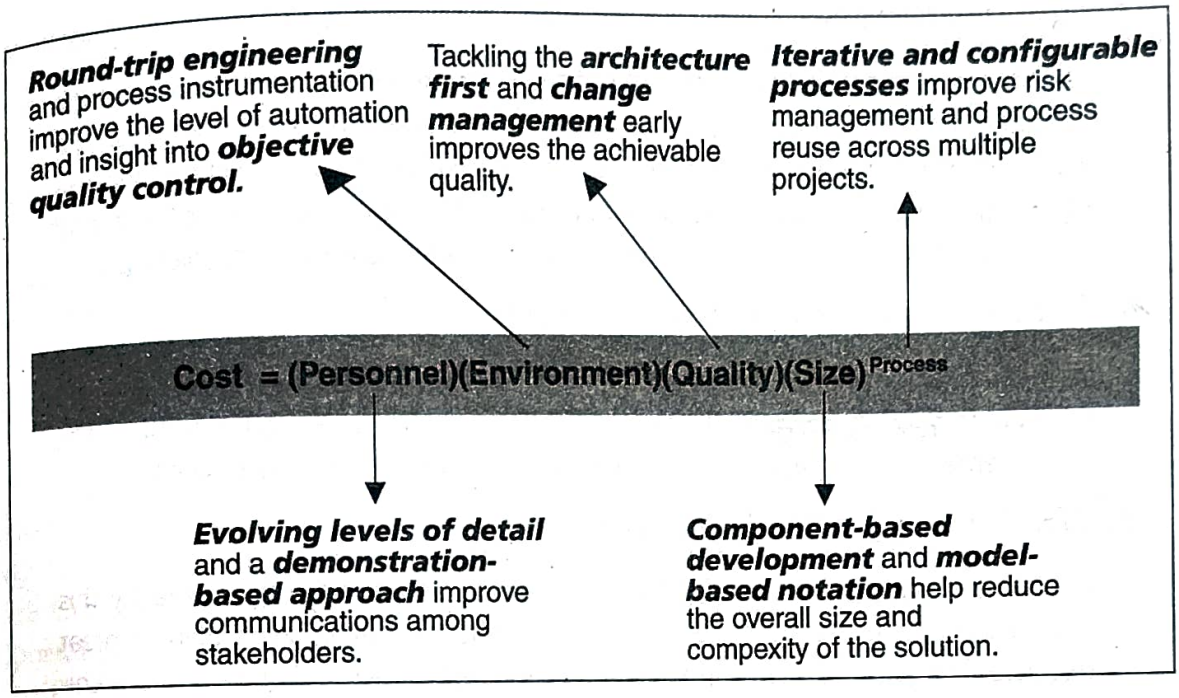


FIGURE 15-4. *Balanced application of modern principles to achieve economic results*

(composed of industry and government practitioners), and a program manager's panel (composed of experienced industry project managers). Each component produced recommendations and results, and reviewed the work of the other components.

The Airlie Software Council was "purposely structured to include highly successful managers of large-scale software projects, internationally recognized authors, prominent consultants, and executives responsible for software development at major companies." One of the Council's products was a set of nine best practices. The Council attempted to focus on the practices that would have the greatest effect in improving the software management discipline for large-scale software projects and controlling the complexities therein.

The nine best practices are described next, with my commentary on how they resonate with the process framework, management disciplines, and top 10 principles that I have recommended. (Quotations are presented in *italics*.)

1. *Formal risk management.*

   ▲ Using an ***iterative process*** that confronts risk is more or less what this is saying.

2. *Agreement on interfaces.*

   ▲ While we may use different words, this is exactly the same intent as my ***architecture-first*** principle. Getting the architecture baselined forces the project to gain agreement on the various external interfaces and the important internal interfaces, all of which are inherent in the architecture.

3. *Formal inspections.*

▲ The assessment workflow throughout the life cycle, along with the other engineering workflows, must balance several different defect removal strategies. The least important strategy, in terms of breadth, should be formal inspection, because of its high costs in human resources and its low defect discovery rate for the critical architectural defects that span multiple components and temporal complexity.

4. *Metric-based scheduling and management.*

▲ This important principle is directly related to my ***model-based notation*** and ***objective quality control*** principles. Without rigorous notations for artifacts, the measurement of progress and quality degenerates into subjective estimates.

5. *Binary quality gates at the inch-pebble level.*

▲ This practice is easy to misinterpret. Too many projects have taken exactly this approach early in the life cycle and have laid out a highly detailed plan at great expense. Three months later, when some of the requirements change or the architecture changes, a large percentage of the detailed planning must be rebaselined. A better approach would be to maintain fidelity of the plan commensurate with an understanding of the requirements and the architecture. Rather than inch pebbles, I recommend establishing milestones in the engineering stage followed by inch pebbles in the production stage. This is the primary message behind my ***evolving levels of detail*** principle.

6. *Programwide visibility of progress versus plan.*

▲ This practice—namely, open communications among project team members—is obviously necessary. None of my principles traces directly to this practice. It seems so obvious, I let it go without saying.

7. *Defect tracking against quality targets.*

▲ This important principle is directly related to my ***architecture-first*** and ***objective quality control*** principles. The make-or-break defects and quality targets are architectural. Getting a handle on these qualities early and tracking their trends are requirements for success.

8. *Configuration management.*

▲ The Airlie Software Council emphasized configuration management as key to controlling complexity and tracking changes to all artifacts. It also recognized that automation is important because of the volume and dynamics of modern, large-scale projects, which make manual methods cost-prohibitive and error-prone. The same reasoning is behind my ***change management*** principle.

9. *People-aware management accountability.*

▲ This is another management principle that seems so obvious, I let it go without saying.

There is significant overlap and commonality of spirit between my top principles and the Airlie Software Council's best practices. However, I think the Council omitted some important principles: configurability and component-based, model-based, demonstration-based development. This omission is surprising, because my rationale for including component-based and model-based principles was to reduce the complexity of development. This is exactly the stated purpose of the Airlie Software Council. The demonstration-based principle is in my top 10 primarily to force integration to occur continuously throughout the life cycle and to promote better stakeholder relationships through a more meaningful medium of communications. Because the Airlie Software Council was focused on a particular domain—namely, large-scale, nationally important systems—configurability was less important.

The two Airlie Software Council practices I would not have included are inspections and binary quality gates at the inch-pebble level. Although they are useful, they are overemphasized in practice, and there are other important principles that should have been included.

CHAPTER 16

Next-Generation Software Economics

Key Points

- ▲ Next-generation software economics should reflect better economies of scale and improved return on investment profiles. These are the real indicators of a mature industry.
- ▲ Further technology advances in round-trip engineering are critical to making the next quantum leap in software economics.
- ▲ Future cost estimation models need to be based on better primitive units defined from well-understood software engineering notations such as the Unified Modeling Language.

Next-generation software economics is being practiced by some advanced software organizations. Many of the techniques, processes, and methods described in this book's process framework have been practiced for several years. However, a mature, modern process is nowhere near the state of the practice for the average software organization. This chapter introduces several provocative hypotheses about the future of software economics. A general structure is proposed for a cost estimation model that would be better suited to the process framework in this book. I think this new approach would improve the accuracy and precision of software cost estimates, and would accommodate dramatic improvements in software economies of scale. Such improvements will be enabled by advances in software development environments. Finally, I look again at Boehm's benchmarks of conventional software project performance and describe, in objective terms, how the process framework should improve the overall software economics achieved by a project or organization.

16.1 NEXT-GENERATION COST MODELS

Software experts hold widely varying opinions about software economics and its manifestation in software cost estimation models: Source lines of code versus function points. Economy of scale versus diseconomy of scale. Productivity measures versus

quality measures. Java versus C++. Object-oriented versus functionally oriented. Commercial components versus custom development. All these topics represent industry debates surrounded by high levels of rhetoric. The passionate overhype or underhype, depending on your perspective, makes it difficult to separate facts from exaggeration. Energetic disagreement is an indicator of an industry in flux, in which many competing technologies and techniques are maturing rapidly. One of the results, however, is a continuing inability to predict with precision the resources required for a given software endeavor. Accurate estimates are possible today, although honest estimates are imprecise. It will be difficult to improve empirical estimation models while the project data going into these models are noisy and highly uncorrelated, and are based on differing process and technology foundations.

Some of today's popular software cost models are not well matched to an iterative software process focused on an architecture-first approach. Despite many advances by some vendors of software cost estimation tools in expanding their repertoire of up-to-date project experience data, many cost estimators are still using a conventional process experience base to estimate a modern project profile. This section provides my perspective on how a software cost model should be structured to best support the estimation of a modern software process. There are cost models and techniques in the industry that can support subsets of this approach. My software cost model is all theory; I have no empirical evidence to demonstrate that this approach will be more accurate than today's cost models. Even though most of the methods and technology necessary for a modern management process are available today, there are not enough relevant, completed projects to back up my assertions with objective evidence.

A next-generation software cost model should explicitly separate architectural engineering from application production, just as an architecture-first process does. The cost of designing, producing, testing, and maintaining the architecture baseline is a function of scale, quality, technology, process, and team skill. There should still be some diseconomy of scale (exponent greater than 1.0) in the architecture cost model because it is inherently driven by research and development-oriented concerns. When an organization achieves a stable architecture, the production costs should be an exponential function of size, quality, and complexity, with a much more stable range of process and personnel influence. The production stage cost model should reflect an economy of scale (exponent less than 1.0) similar to that of conventional economic models for bulk production of commodities. Figure 16-1 summarizes an hypothesized cost model for an architecture-first development process.

Next-generation software cost models should estimate large-scale architectures with economy of scale. This implies that the process exponent during the production stage will be less than 1.0. My reasoning is that the larger the system, the more opportunity there is to exploit automation and to reuse common processes, components, and architectures.

$$\text{Effort} = F(T_{Arch}, S_{Arch}, Q_{Arch}, P_{Arch}) + F(T_{App}, S_{App}, Q_{App}, P_{App})$$

$$\text{Time} = F(P_{Arch}, \text{Effort}_{Arch}) + F(P_{App}, \text{Effort}_{App})$$

where:
T = technology parameter (environment automation support)
S = scale parameter (such as use cases, function points, source lines of code)
Q = quality parameter (such as portability, reliability, performance)
P = process parameter (such as maturity, domain experience)

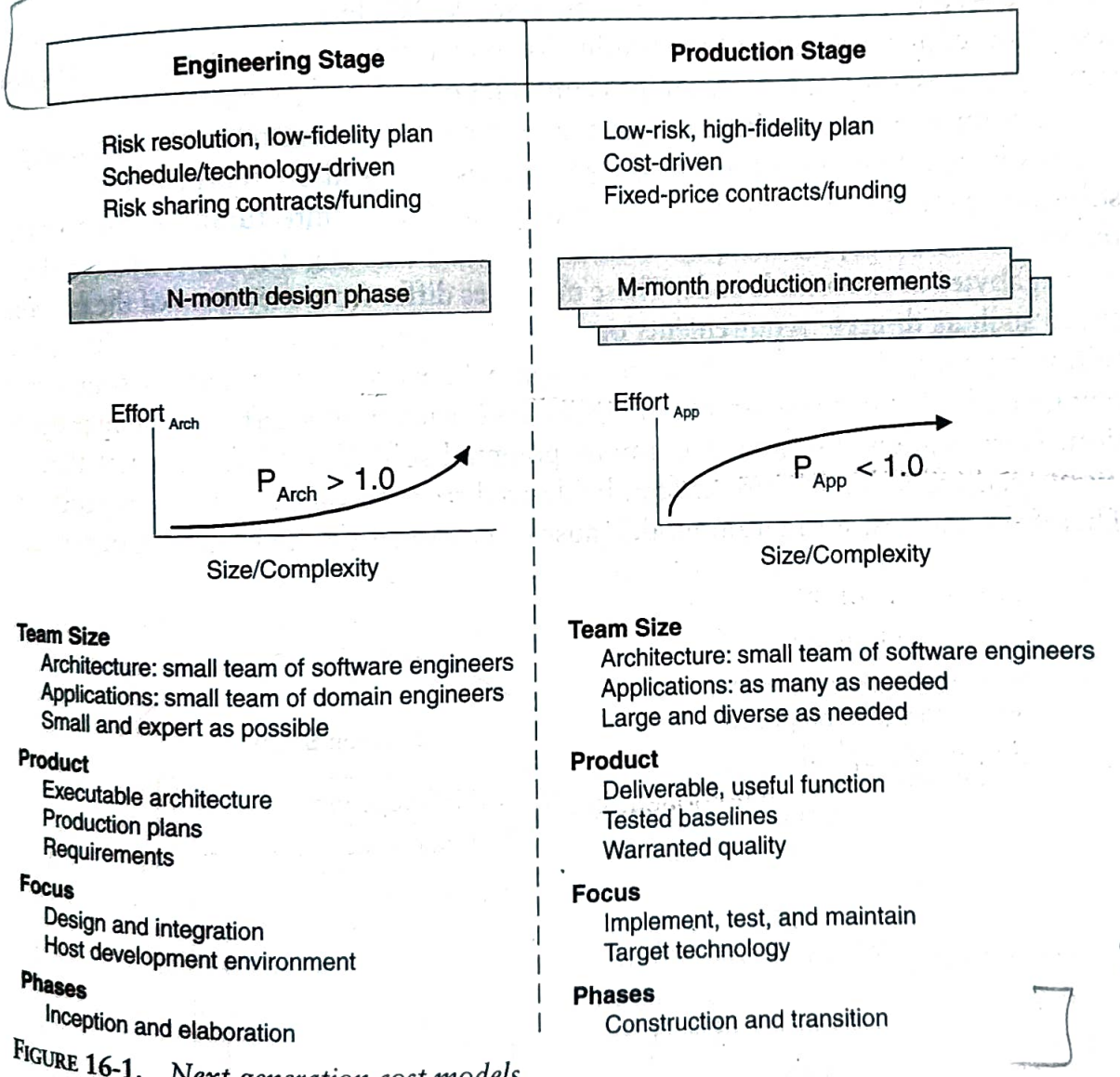


FIGURE 16-1. *Next-generation cost models*

In the conventional process, the minimal level of automation that supported the overhead activities of planning, project control, and change management led to labor-intensive workflows and a diseconomy of scale. This lack of management automation was as true for multiple-project, line-of-business organizations as it was for individual projects. Next-generation environments and infrastructures are moving to automate

and standardize many of these management activities, thereby requiring a lower percentage of effort for overhead activities as scale increases.

Reusing common processes across multiple iterations of a single project, multiple releases of a single product, or multiple projects in an organization also relieves many of the sources of diseconomy of scale. Critical sources of scrap and rework are eliminated by applying precedent experience and mature processes. Establishing trustworthy plans based on credible project performance norms and using reliable components reduce other sources of scrap and rework. While most reuse of components results in reducing the size of the production effort, the reuse of processes, tools, and experience has a direct impact on the economies of scale.

Another important difference in this cost model is that architectures and applications have different units of mass (scale versus size) and are representations of the solution space. Scale might be measured in terms of architecturally significant elements (classes, components, processes, nodes), and size might be measured in SLOC or megabytes of executable code. These measures differ from measures of the problem space such as discrete requirements or use cases. The problem space description certainly drives the definition of the solution space. However, there are many solutions to any given problem, as illustrated in Figure 16-2, each with a different value proposition. Cost is a key discriminator among potential solutions. Cost estimates that are more accurate and more precise can be derived from specific solutions to problems. Therefore, the cost estimation model must be governed by the basic parameters of a

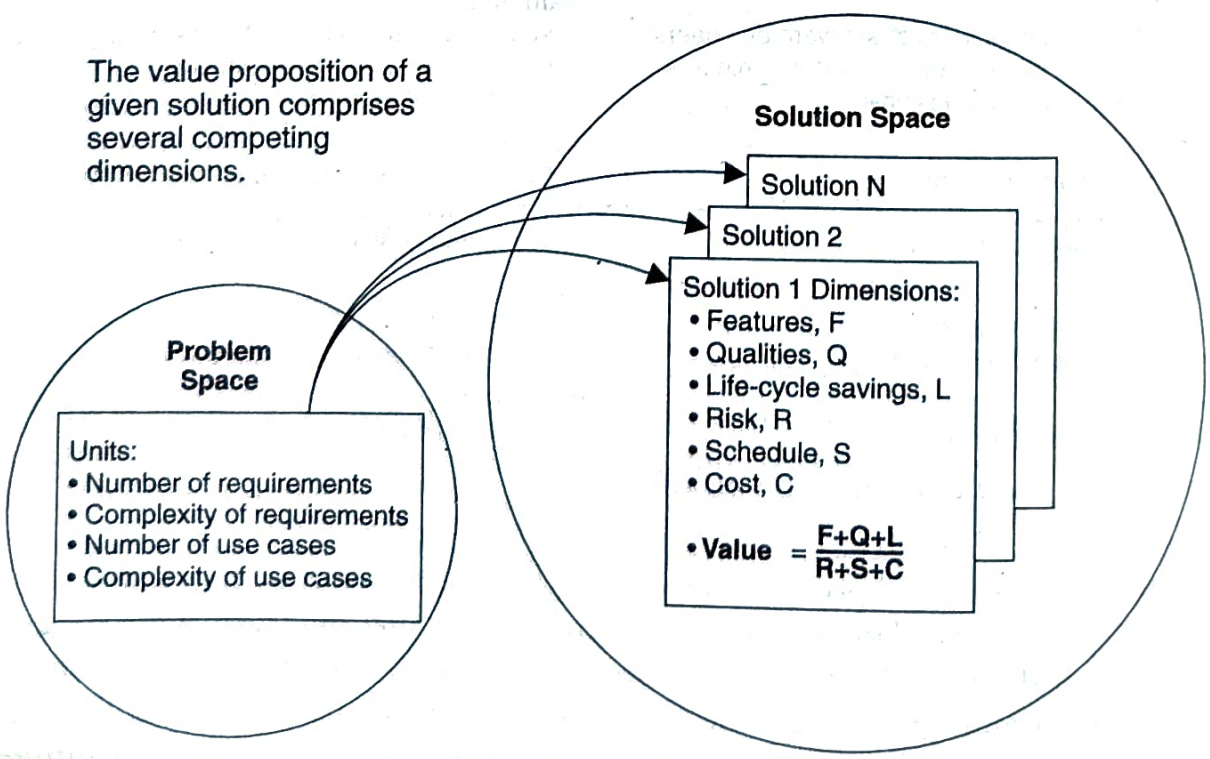


FIGURE 16-2. *Differentiating potential solutions through cost estimation*

candidate solution. If none of the value propositions is an acceptable solution to the problem, further candidate solutions need to be pursued or the problem statement needs to change.

The debate between function point zealots and source line zealots is a good indicator of the need for measures of both scale and size. I think function points are more accurate at quantifying the scale of the architecture required, while SLOC more accurately depicts the size of the components that make up the total implementation. The beauty of using SLOC is that collection can be easily automated and precision can be easily achieved. However, the accuracy of SLOC as a measure of size is ambiguous and can lead to misinterpretation when SLOC is used in absolute comparisons among different projects and organizations. This is particularly true in the early phases of projects if SLOC is used to represent scale. Many projects have used SLOC as a successful measure of size in the later phases of the life cycle, when the most important measures are the relative changes from month to month as the project converges on releasable versions.

The value of function points is that they are better at depicting the overall scale of the solution, independently of the actual size and implementation language of the final realization. Function points are not easily extracted from any rigorous representation format, however, so automation and change tracking are difficult or ambiguous.

A rigorous notation for design artifacts is a necessary prerequisite to improvements in the fidelity with which the scale of a design can be estimated. In the future, I expect there will be an opportunity to automate a new measure of scale derived directly from design representations in UML.

I expect two major improvements in next-generation software cost estimation models:

1. Separation of the engineering stage from the production stage will force estimators to differentiate between architectural scale and implementation size. This will permit greater accuracy and more-honest precision in life-cycle estimates.
2. Rigorous design notations such as UML will offer an opportunity to define units of measure for scale that are more standardized and therefore can be automated and tracked. These measures can also be traced more straightforwardly into the costs of production.

Quantifying the scale of the software architecture in the engineering stage is an area ripe for research. Over the next decade, two breakthroughs in the software process seem possible, both of them realized through technology advances in the supporting environment. The first breakthrough would be the availability of integrated tools that automate the transition of information between requirements, design, implementation, and deployment elements. These tools would allow more comprehensive

4. *Software development and maintenance costs are primarily a function of the number of source lines of code.*

   ▲ This metric says that the size of the product is the primary cost driver, and the fundamental unit of size is a line of code. While this was obvious in previous generations of software technology, it is becoming much less obvious in today's component-based technologies. Commercial components, reuse, and automatic code generators can seriously pollute the meaning of a source line of code. Construction costs will still be driven by the complexity of the bill of materials. The use of more components, more types of components, more sources of components, and more custom components will necessitate more integration labor and will drive up costs. The use of fewer components, fewer types, fewer sources, and more industrial-strength tooling will drive down costs. Unfortunately, the component industry is still too immature to agree on standards for a bill of materials that could improve the fidelity of its cost estimations. Therefore, the next-generation cost models should become less sensitive to the number of source lines and more sensitive to the discrete numbers of components and their ease of integration.

5. *Variations among people account for the biggest differences in software productivity.*

   ▲ For any engineering venture in which intellectual property is the real product, the dominant productivity factors will be personnel skills, teamwork, and motivations. To the extent possible, a modern process encapsulates the requirements for high-leverage people in the engineering stage, when the team is relatively small. The production stage, when teams typically are much larger, should then operate with far less dependency on scarce expertise.

6. *The overall ratio of software to hardware costs is still growing. In 1955, it was 15:85; in 1985, 85:15.*

   ▲ I'm not sure what this metric looks like today. The popularity of the personal computer and the differences in post-1985 versus pre-1985 software costs, particularly personal computer software tools, have undoubtedly changed the relationship. The main impact of this metric on software economics is that hardware continues to get cheaper. Processing cycles, storage, and network bandwidth continue to offer new opportunities for automation. Consequently, software environments are playing a much more important role. From a modern process perspective, I can see the environment doing much more of the bookkeeping and analysis activities that were previously done by humans. Configuration control and quality assurance analyses are already largely automated, and the next frontier is automated production and automated testing.

7. *Only about 15% of software development effort is devoted to programming.*

▲ In the past 10 years there has been a noticeable shift away from investments in languages and compilers, Java and Ada 95 notwithstanding. Modern technology investments have transitioned into process maturity (for example, the SEI CMM), visual modeling (such as UML), automated software quality (such as test automation), components (such as ActiveX, Java, and CORBA), configuration management, metrics, and other aspects of software engineering. The amount of programming that goes on in a software development project is still roughly 15%. The difference is that modern projects are programming at a much higher level of abstraction. An average staff-month of programming produced perhaps 200 machine instructions in the 1960s and 1,000 machine instructions in the 1970s and 1980s. Programmer productivity in the 1990s can produce tens of thousands of machine instructions in a single month, even though only a few hundred human-generated source lines may be produced.

8. *Software systems and products typically cost 3 times as much per SLOC as individual software programs. Software-system products (i.e., system of systems) cost 9 times as much.*

▲ This diseconomy of scale should be greatly relieved with a modern process and modern technologies. Under certain circumstances—such as a software line of business producing discrete, customer-specific software systems with a common architecture, common environment, and common process—an economy of scale is achievable.

9. *Walkthroughs catch 60% of the errors.*

▲ I have emphasized the need for this metric to be banished from the top 10. Human inspections and walkthroughs will not expose the critical issues; they will only help resolve them. This metric should be replaced by the following: While the environment catches most of the first-level inconsistencies and errors, the really important architectural issues can be exposed only through demonstration and early testing and resolved through human scrutiny.

10. *80% of the contribution comes from 20% of the contributors.*

▲ This relationship is timeless and constitutes the background philosophy to be applied throughout the planning and conduct of a modern software management process.

CHAPTER 17

Modern Process Transitions

Key Points

- The transition to modern software processes and technologies necessitates several culture shifts that will not always be easy to achieve.
- Lessons learned in transitioning organizations to a modern process have exposed several recurring themes of success that represent important culture shifts from conventional practice.
- A significant transition should be attempted on a significant project. Pilot projects do not generally attract top talent, and top talent is crucial to the success of any significant transition.

Successful software management is hard work. Technical breakthroughs, process breakthroughs, and new tools will make it easier, but management discipline will continue to be the crux of software project success. New technological advances will be accompanied by new opportunities for software applications, new dimensions of complexity, new avenues of automation, and new customers with different priorities. Accommodating these changes will perturb many of our ingrained software management values and priorities. However, striking a balance among requirements, designs, and plans will remain the underlying objective of future software management endeavors, just as it is today.

The software management framework I have presented in this book is not revolutionary; numerous projects have been practicing some of these disciplines for years. However, many of the techniques and disciplines suggested herein will necessitate a significant paradigm shift. Some of these changes will be resisted by certain stakeholders or by certain factions within a project or organization. It is not always easy to separate cultural resistance from objective resistance. This chapter summarizes some of the important culture shifts to be prepared for in order to avoid as many sources of friction as possible in transitioning successfully to a modern process.

17.1 CULTURE SHIFTS

Several culture shifts must be overcome to transition successfully to a modern software management process. For some of these adjustments, it will be difficult to distinguish between objective opposition and stubborn resistance. Nevertheless, there are general indications of a successful transition to a modern culture. This section discusses several of the rough indicators to look for in order to differentiate projects that have made a genuine cultural transition from projects that have only put up a facade. Many of these indicators are derived directly from the process framework described in earlier chapters; others are second-order effects.

- **Lower level and mid-level managers are performers.** There should be no "pure managers" in an organization or suborganization with 25 or fewer people. The need for pure managers arises only when personnel resources exceed this level. Hands-on management skills vary, but competent managers typically spend much of their time performing, especially with regard to understanding the status of the project firsthand and developing plans and estimates. Above all, the person managing an effort should plan it. This does not mean the manager should approve the plan; it means the manager should participate in developing it. In independent project assessments I have performed, a good indicator of trouble ahead is a manager who did not author the plan nor take ownership of it. The stakeholders affected by this transition are software project managers.
- **Requirements and designs are fluid and tangible.** The conventional process focused too much on producing documents that attempted to describe the software product and focused too little on producing tangible increments of the products themselves. Major milestones were defined solely in terms of specific documents. Development organizations for large contractual projects were driven to produce tons of paper in order to meet milestones and receive progress payments, rather than spend their energy on tasks that would have reduced risk and produced quality software. An iterative process requires actual construction of a sequence of progressively more comprehensive systems that demonstrate the architecture, enable objective requirements negotiations, validate the technical approach, and address resolution of key risks. Ideally, all stakeholders would be focused on these "real" milestones, with incremental deliveries of useful functionality rather than speculative paper descriptions of the end-item vision. The transition to a less document-driven environment will be embraced by the engineering teams; it will probably be resisted by traditional contract monitors.
- **Ambitious demonstrations are encouraged.** The purpose of early life-cycle demonstrations is to expose design flaws, not to put up a facade. Stake-

holders must not overreact to early mistakes, digressions, or immature designs. Evaluation criteria in early release plans are goals, not requirements. If early engineering obstacles are overemphasized, development organizations will set up future iterations to be less ambitious. On the other hand, stakeholders should not tolerate lack of followthrough in resolving issues. If negative trends are not addressed with vigor, they can cause serious downstream perturbations. Open and attentive follow-through is necessary to resolve issues. The management team is most likely to resist this transition (especially if the project was oversold), because it will expose any engineering or process issues that were easy to hide using the conventional process. Customers, users, and the engineering team will embrace this transition for exactly the same reason.

- **Good and bad project performance is much more obvious earlier in the life cycle.** In an iterative development, success breeds success, and early failures are extremely risky to turn around. Real-world project experience has shown time and again that it is the early phases that make or break a project. It is therefore of paramount importance to ensure that the very best team possible performs the planning and architecture phases. If these phases are done right and with good teams, projects can be completed successfully by average teams evolving the applications into the final product. If the planning and architecture phases are not performed adequately, all the expert programmers and testers in the world probably will not achieve success. No one should resist early staffing with the right team. However, most organizations have scarce resources for these sorts of early life-cycle roles and are hesitant to make the necessary staff allocations.
- **Early increments will be immature.** External stakeholders, such as customers and users, cannot expect initial deliveries to perform up to specification, to be complete, to be fully reliable, or to have end-target levels of quality or performance. On the other hand, development organizations must be held accountable for, and demonstrate, tangible improvements in successive increments. The trends will indicate convergence toward specification. Objectively quantifying changes, fixes, and upgrades will indicate the quality of the process and environment for future activities. Customers and users will have difficulty accepting the flaws of early releases, although they should be impressed by later increments. Management and the development team will accept immaturity as a natural part of the process.
- **Artifacts are less important early, more important later.** It is a waste of time to worry about the details (traceability, thoroughness, and completeness) of the artifact sets until a baseline is achieved that is useful enough and stable enough to warrant time-consuming analyses of these quality factors.

Otherwise, a project will squander early engineering cycles and precious resources adding content and quality to artifacts that may quickly become obsolete. While the development team will embrace this transition wholeheartedly, traditional contract monitors will resist the early de-emphasis on completeness.

- **Real issues are surfaced and resolved systematically.** Successful projects recognize that requirements and designs evolve together, with continuous negotiation, trade-off, and bartering toward best value, rather than blindly adhering to an ambiguous contract statement. On a healthy project that is making progress, it should be easy to differentiate between real and apparent issues. Depending on the situation, this culture shift could affect almost any team.
- **Quality assurance is everyone's job, not a separate discipline.** Many organizations have a separate group called quality assurance. I am generally against the concept of separate quality assurance activities, teams, or artifacts. Quality assurance should be woven into every role, every activity, every artifact. True quality assurance is measured by tangible progress and objective data, not by checklists, meetings, and human inspections. The software project manager or designee should assume the role of ensuring that quality assurance is properly integrated into the process. The traditional policing by a separate team of inspectors is replaced by the self-policing teamwork of an organization with a mature process, common objectives, and common incentives. Traditional managers and quality assurance personnel will resist this transition. Engineering teams will embrace it.
- **Performance issues arise early in the life cycle.** Early performance issues have surfaced on almost every successful project I know of. These issues are a sign of an immature design but a mature design process. Stakeholders will usually be concerned over early performance issues. Development engineers will embrace the emphasis on early demonstrations and the ability to assess and evaluate performance trade-offs in subsequent releases.
- **Investments in automation are necessary.** Because iterative development projects require extensive automation, it is important not to underinvest in the capital environment. It is also important for stakeholders to acquire an integrated environment that permits efficient participation in an iterative development. Otherwise, interactions with the development organization will degenerate to paper exchange and many of the issues of the conventional process. These investments may be opposed by organization managers overly focused on near-term financial results or by project personnel

who favor the preference of the individual project over the global solution that serves both the project and the organization goals.

- **Good software organizations should be more profitable.** In the commercial software domain, this is not an issue. In most of the software contracting domain, especially government contracts, it is definitely an issue. As part of the adversarial nature of the acquisition and contracting process, there is considerable focus on ensuring that contractor profits are within a certain acceptable range (typically 5% to 15%). Occasionally, excellent contractor performance, good value engineering, or significant reuse results in potential contractor profit margins in excess of the acceptable initial bid. As soon as customers (or their users or engineering monitors) become aware of such a trend, it is inevitable that substantial pressure will be exerted to apply these "excess" resources to out-of-scope changes until the margin is back in the acceptable range.

As a consequence, the simple profit motive that underlies commercial transactions and incentivizes efficiency is replaced by complex contractual incentives (and producer-consumer conflicts) that are usually suboptimal. Very frequently, contractors see no economic incentive to implement major cost savings, and certainly there is little incentive to take risks that may have a large return. On the other hand, contractors can easily consume large amounts of money (usually at a small profit margin) without producing results and with little accountability for poor performance.

For the software industry to prosper, good contractors should be rewarded (more profit) and bad contractors should be punished (less profit). A customer who gets a good product at a reasonable price should be happy that the contractor also made a good profit. Allowing contractors who perform poorly to continue doing so is not good for anyone. This is one area in which the commercial domain is far more effective than the government contracting domain.

17.2 DENOUEMENT

In summary, the conventional software process was characterized by the following:

- Sequentially transitioning from requirements to design to code to test
- Achieving 100% completeness of each artifact at each life-cycle stage
- Treating all requirements, artifacts, components, and so forth, as equals
- Achieving high-fidelity traceability among all artifacts at each stage in the life cycle

A modern iterative development process framework is characterized by the following:

- Continuous round-trip engineering from requirements to test at evolving levels of abstraction
- Achieving high-fidelity understanding of the drivers (the 20%) as early as practical
- Evolving the artifacts in breadth and depth based on risk management priorities
- Postponing completeness and consistency analyses until later in the life cycle

A modern process framework attacks the primary sources of the diseconomy of scale inherent in the conventional software process. Figure 17-1 illustrates the next generation of software project performance by depicting the development progress versus time, where progress is defined as percent coded (demonstrable in its target form). (The figure follows the same presentation format as Figures 1-2 and 15-1.)

My goal in this book has been to explain how to move onto the upper, shaded region, with a modern process supported by an advanced, fully integrated environment and a component-based architecture. Organizations that succeed should be capable of deploying software products that are constructed largely from existing components in 50% less time, with 50% fewer development resources, and maintained by teams 50% the size of those required by today's systems.

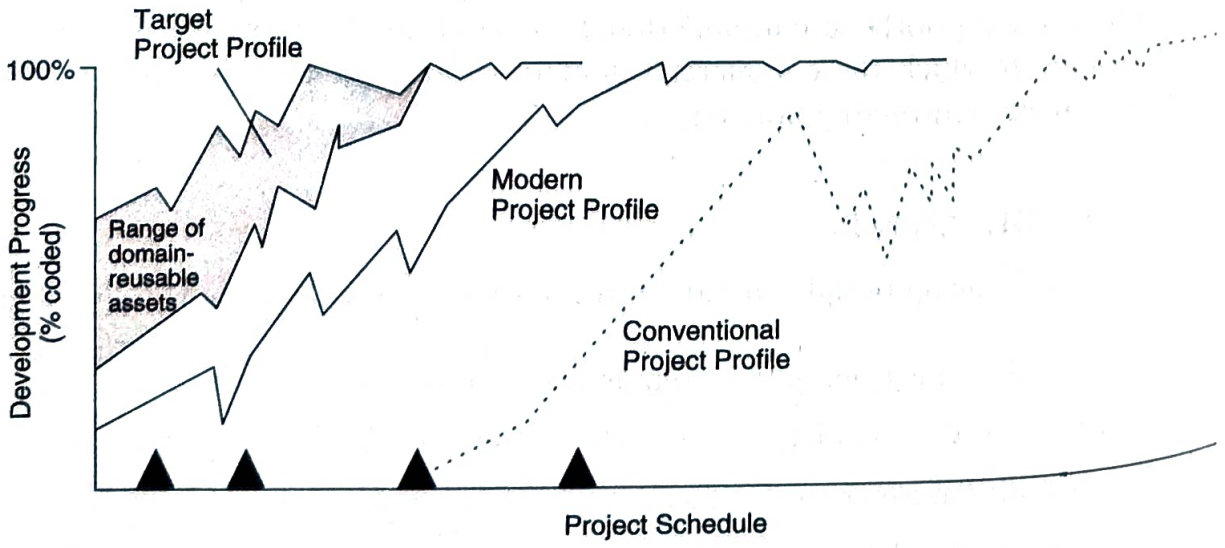


FIGURE 17-1. *Next-generation project performance*

As an organization transitions to new techniques and technologies, there is always apprehension and concern about failing. Maintaining the status quo and relying on existing methods is usually considered the safest path. In the software industry, where most organizations succeed on only a small percentage of their projects, maintaining the status quo is not always safe. When an organization decides to make a transition, these two pieces of conventional wisdom are usually offered by internal champions as well as external change agents: (1) Pioneer any new techniques on a small pilot program. (2) Be prepared to spend more resources—money and time—on your first project that makes the transition. I see both recommendations as counterproductive.

Small pilot programs outside the mainstream have their place, but they rarely achieve any paradigm shift of consequence. Trying a new little technique, tool, or method on a very rapid, small-scale effort—less than 3 months, say, and only a few people—can frequently show good results, initial momentum, or proof of concept. The problem with pilot programs is that they are almost never on the critical path of the organization. Consequently, they do not merit "A" players, adequate resources, or management attention.

The most successful organizational paradigm shifts I have seen resulted from sets of circumstances similar to these: The organizations took their most critical project and highest caliber personnel, gave them adequate resources, *and* demanded better results. If, on the other hand, an organization expects a new method, tool, or technology to have an adverse impact on the results of the trailblazing project, that expectation is almost certain to come true. Why? Because no organization manager would knowingly cause an adverse impact on the most important projects in the organization, and that is where the organization's best people will be assigned. Therefore, the trailblazing project will be a noncritical project staffed with noncritical personnel of whom less is expected. This low expectation is often a self-fulfilling prophecy.

A better way to transition to a more mature iterative development process that supports automation technologies and modern architectures is to take the following shot:

- *Ready.* Do your homework. Analyze modern approaches and technologies. Define (or improve, or optimize) your process. Support it with mature environments, tools, and components. Plan thoroughly.
- *Aim.* Select a critical project. Staff it with the right team of complementary resources and demand improved results.
- *Fire.* Execute the organizational and project-level plans with vigor and follow-through.

APPENDIX D

CCPDS-R Case Study

This appendix presents a detailed case study of a successful software project that followed many of the techniques presented in this book. *Successful* here means on budget, on schedule, and satisfactory to the customer. The Command Center Processing and Display System-Replacement (CCPDS-R) project was performed for the U.S. Air Force by TRW Space and Defense in Redondo Beach, California. The entire project included systems engineering, hardware procurement, and software development, with each of these three major activities consuming about one-third of the total cost. The schedule spanned 1987 through 1994.

The software effort included the development of three distinct software systems totaling more than one million source lines of code. This case study focuses on the initial software development, called the Common Subsystem, for which about 355,000 source lines were developed. The Common Subsystem effort also produced a reusable architecture, a mature process, and an integrated environment for efficient development of the two software subsystems of roughly similar size that followed. This case study therefore represents about one-sixth of the overall CCPDS-R project effort.

Although this case study does not coincide exactly with the management process presented in this book nor with all of today's modern technologies, it used most of the same techniques and was managed to the same spirit and priorities. TRW delivered

Key Points

- An objective case study is a true indicator of a mature organization and a mature project process. The software industry needs more case studies like CCPDS-R.
- The metrics histories were all derived directly from the artifacts of the project's process. These data were used to manage the project and were embraced by practitioners, managers, and stakeholders.
- CCPDS-R was one of the pioneering projects that practiced many modern management approaches.
- This appendix provides a practical context that is relevant to the techniques, disciplines, and opinions provided throughout this book.

the system on budget and on schedule, and the users got more than they expected. TRW was awarded the Space and Missile Warning Systems Award for Excellence in 1991 for "continued, sustained performance in overall systems engineering and project execution." A project like CCPDS-R could be developed far more efficiently today. By incorporating current technologies and improved processes, environments, and levels of automation, this project could probably be built today with equal quality in half the time and at a quarter of the cost.

D.1 CONTEXT FOR THE CASE STUDY

I worked full time on the CCPDS-R project for 6 years, so this appendix is written from firsthand experience. My responsibilities included managing the development of the foundation technologies, developing the technical and cost proposals, conducting the software engineering exercise, and managing the software engineering activities through the early operational capability milestone.

I have tried to provide an accurate portrayal of the CCPDS-R project. While the data presented are mostly historical fact, all the subjective comments and value judgments are mine. The data were derived from published papers, internal TRW guidebooks, contract deliverable documents—all available from the actual artifacts of the CCPDS-R project—and my own personal experience. In a few minor cases, I have edited the data to remove unnecessary precision and eliminate inconsistencies within source documents produced at different points in the life cycle. My goal was to produce a relatively consistent description while excluding some of the minutia that would require detailed and irrelevant explanation.

Although the software industry can claim many successful projects (not enough, but many), good case studies are lacking. There are very few well-documented projects with objective descriptions of what worked, what didn't, and why. This was one of my primary motivations for providing the level of detail contained in this appendix. It is heavy in project-specific details, approaches, and results, for three reasons:

1. Generating the case study wasn't much work. CCPDS-R is unique in its detailed and automated metrics approach. All the data were derived directly from the historical artifacts of the project's process.
2. This sort of objective case study is a true indicator of a mature organization and a mature project process. The absolute values of this historical perspective are only marginally useful. However, the trends, lessons learned, and relative priorities are distinguishing characteristics of successful software development.

3. Throughout previous chapters, many management and technical approaches are discussed generically. This appendix provides in a real-world example at least one relevant benchmark of performance.

My comments on relevance with the techniques, disciplines, and opinions discussed in previous chapters are provided in shaded boxes.

D.2 COMMON SUBSYSTEM OVERVIEW

The CCPDS-R project produced a large-scale, highly reliable command and control system that provides missile warning information used by the National Command Authority. The procurement agency was Air Force Systems Command Headquarters, Electronic Systems Division, at Hanscom Air Force Base, Massachusetts. The primary user was US Space Command, and the full-scale development contract was awarded to TRW's Systems Integration Group in 1987. The CCPDS-R contract called for the development of three subsystems:

1. The Common Subsystem was the primary missile warning system within the Cheyenne Mountain Upgrade program. It required about 355,000 source lines of code, had a 48-month software development schedule, and laid the foundations for the subsystems that followed (reusable components, tools, environment, process, procedures). The Common Subsystem included a primary installation in Cheyenne Mountain, with a backup system deployed at Offutt Air Force Base, Nebraska.
2. The Processing and Display Subsystem (PDS) was a scaled-down missile warning display system for all nuclear-capable commanders-in-chief. The PDS software (about 250,000 SLOC) was fielded on remote, read-only workstations that were distributed worldwide.
3. The STRATCOM Subsystem (about 450,000 SLOC) provided both missile warning and force management capability for the backup missile warning center at the command center of the Strategic Command.

Overall Software Acquisition Process

The CCPDS-R acquisition included two distinct phases: a concept definition (CD) phase and a full-scale development (FSD) phase. The CD phase proposal was competed for by five major bidders, and two firm-fixed-price contracts of about $2 million each were awarded. The winning contractors also invested their own discretionary resources to discriminate themselves with the best-value FSD phase proposal. Figure D-1 summarizes the overall acquisition process and the products of each phase.

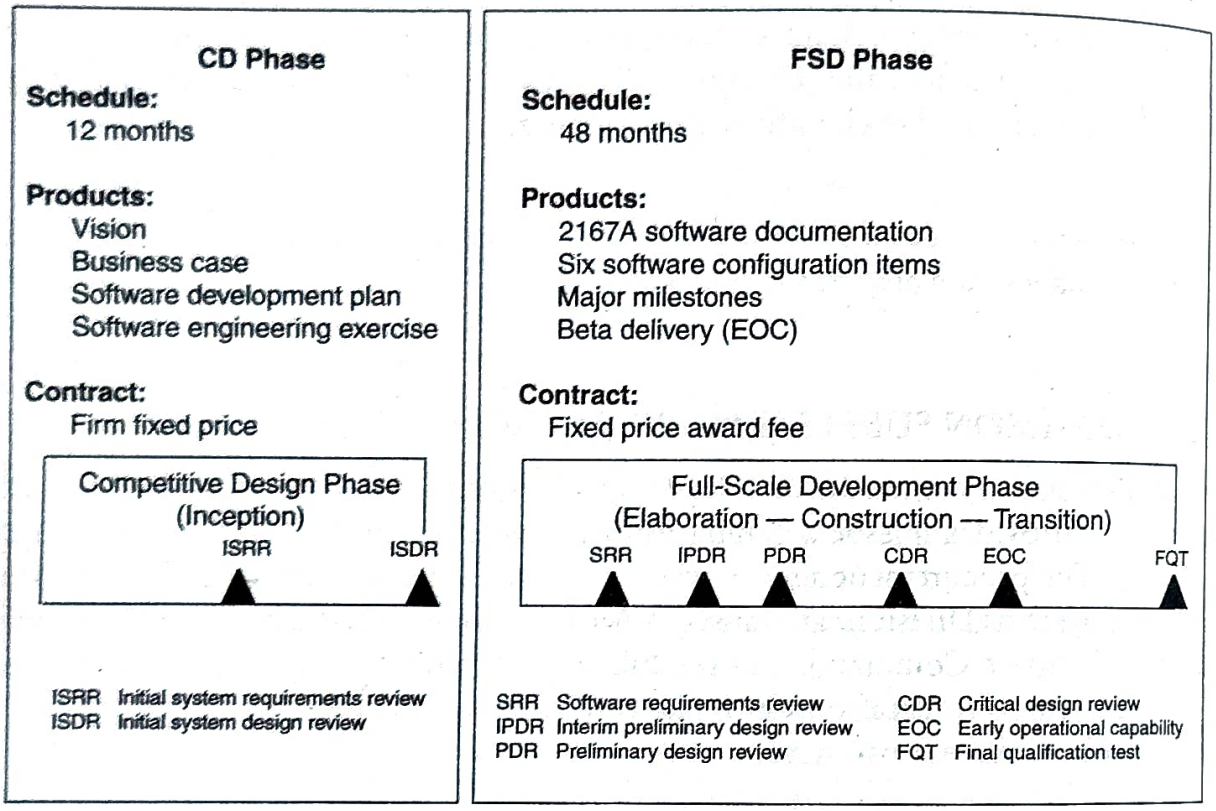


FIGURE D-1. *CCPDS-R life-cycle overview*

The CD phase was very similar in intent to the inception phase. The primary products were a system specification (a vision document), an FSD phase proposal (a business case, including the technical approach and a fixed-price-incentive and award-fee cost proposal), and a software development plan. The CD phase also included a system design review, technical interchange meetings with the government stakeholders (customer and user), and several contract-deliverable documents. These events and products enabled the FSD source selection to be based on demonstrated performance of the contractor-proposed team as well as the FSD proposal.

From a software perspective, there was one additional source selection criterion included in the FSD proposal activities: a software engineering exercise. This was a unique but very effective approach for assessing the abilities of the two competing contractors to perform software development. The Air Force was extremely concerned with the overall software risk of this project: Recent projects had demonstrated dismal software development performance. The Air Force acquisition authorities had also been frustrated with previous situations in which a contractor's crack proposal team was not the team committed to perform after contract award, and contractor proposals exaggerated their approaches or capabilities beyond what they could deliver.

CCPDS-R was also a very large software development activity and was one of the first projects to use the Ada programming language. There was serious concern

that the Ada development environments, contractor processes, and contractor training programs might not be mature enough to use on a full-scale development effort. The purpose of the software engineering exercise was to demonstrate that the contractor's proposed software process, Ada environment, and software team were in place, were mature, and were demonstrable.

The software engineering exercise occurred immediately after the FSD proposals were submitted. The customer provided both bidders with a simple two-page specification of a "missile warning simulator." This simulator had some of the same fundamental requirements as the CCPDS-R full-scale system, including a distributed architecture, a flexible user interface, and the basic processing scenarios of a simple CCPDS-R missile warning thread. The exercise requirements included the following:

- Use the proposed software team.
- Use the proposed software development techniques and tools.
- Use the FSD-proposed software development plan.
- Conduct a mock design review with the customer 23 days after receipt of the specification.

The software engineering exercise would provide objective evidence of the credibility of each contractor's proposed software development approach.

The results produced by TRW's CCPDS-R team were impressive. They demonstrated to the customer that the team was prepared, credible, and competent at conducting the proposed software approach. Approximately 12 staff-months were expended in the effort (12 people full-time for 23 days).

A detailed plan was established that included an activity network, responsibility assignments, and expected results for tracking progress. The plan included two architecture iterations and all the milestones and artifacts proposed in the software development plan. The exercise produced the following results:

- Four primary use cases were elaborated and demonstrated.
- A software architecture skeleton was designed, prototyped, and documented, including two executable, distributed processes; five concurrent tasks (separate threads of control); eight components; and 72 component-to-component interfaces.
- A total of 4,163 source lines of prototype components were developed and executed. Several thousand lines of reusable components were also integrated into the demonstration.
- Three milestones were conducted and more than 30 action items resolved.

- Production of 11 documents (corresponding to the proposed artifacts) demonstrated the automation inherent in the documentation tools.
- The Digital Equipment Corporation VAX/VMS tools, Rational R1000 environment, LaTeX documentation templates, and several custom-developed tools were used.
- Several needed improvements to the process and the tools were identified. The concept of evolving the plan, requirements, process, design, and environment at each major milestone was considered potentially risky but was implemented with rigorous change management.

This exercise proved to be a discriminating factor in the CCPDS-R contract award. TRW had proposed an architecture-first, demonstration-based approach and had demonstrated its operational concept successfully under realistic, albeit small-scale and accelerated, conditions. Despite submitting a bid that was more than 20% higher than that of their competitor, TRW's approach was selected as the best value and lowest risk. Award of the contract to TRW was due, in large part, to successful performance on the software engineering exercise and TRW's ability to demonstrate a much more credible, lower risk process under realistic conditions.

The software engineering exercise served the same purpose as an SEI Software Capability Evaluation (Appendix E). Each bidder's proposal provided a software development plan—the "say what you do" part of an organizational process. The exercise demonstrated that the proposing organization could perform as advertised.

D.3 PROJECT ORGANIZATION

In preparing for the CCPDS-R project, TRW placed a strong emphasis on evolving the right team. The CD phase team represented the essence of the architecture team (Section 11.2), which is responsible for an efficient engineering stage. This team had the following primary responsibilities:

- Analyze and specify the project requirements
- Define and develop the top-level architecture
- Plan the FSD phase software development activities
- Configure the process and development environment
- Establish trust and win-win relationships among the stakeholders

The CD phase team was small and expert, with little, if any, organizational hierarchy. One of its exceptional attributes was its complement of talent. All necessary skills were covered, and there was very little competition among personnel.

The FSD phase team was formed by transitioning many of the CD phase team members into leadership positions and expanding the number of personnel to the necessary levels for full-scale development. Figure D-2 illustrates the software organization evolution and FSD software responsibilities.

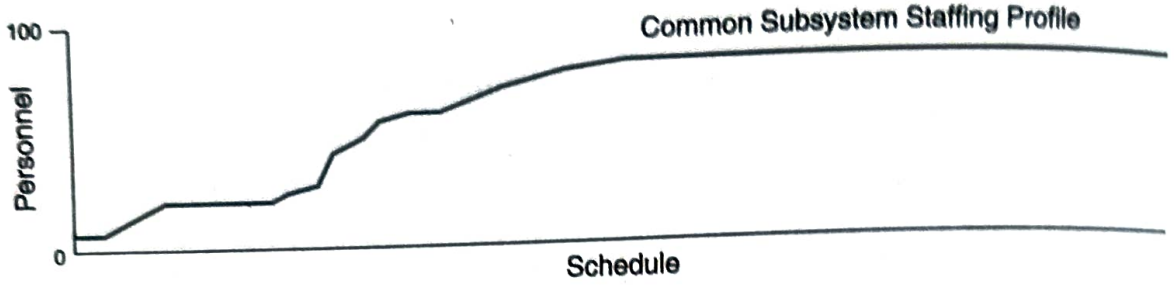
The organizational structure and responsibilities for CCPDS-R were very similar to those recommended in Figure 11-2. The staffing levels evolved as prescribed in Table 10-2.

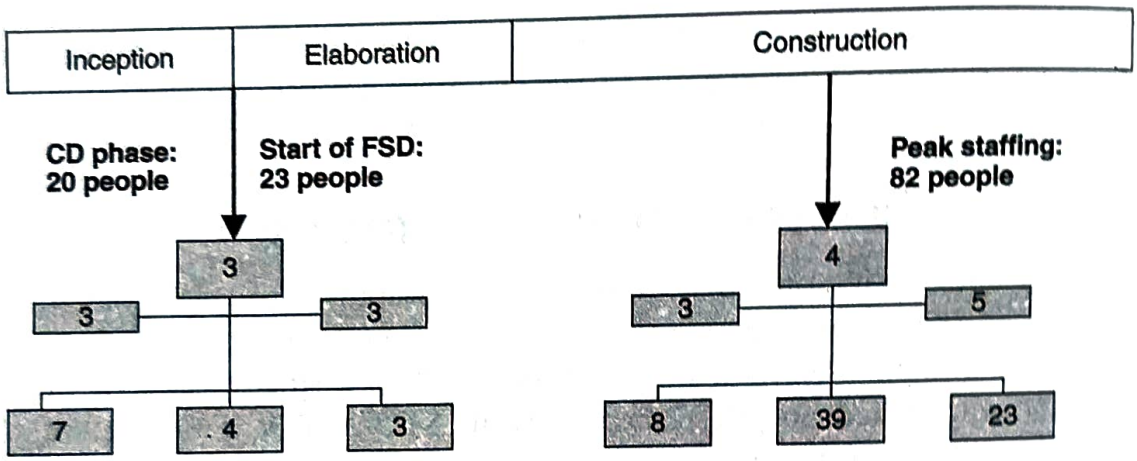
D.4 COMMON SUBSYSTEM PRODUCT OVERVIEW

The Common Subsystem software comprised six computer software configuration items (CSCIs). (*CSCI* is government jargon for a set of components that is managed, configured, and documented as a unit and allocated to a single team for development.) CSCIs are defined and described in DOD-STD-2167A [DOD, 1988]. The CSCIs were identified as follows:

1. Network Architecture Services (NAS). This foundation middleware provided reusable components for network management, interprocess communications, initialization, reconfiguration, anomaly management, and instrumentation of software health, performance, and state. This CSCI was designed to be reused across all three CCPDS-R subsystems.
2. System Services (SSV). This CSCI comprised the software architecture skeleton, real-time data distribution, global data types, and the computer system operator interface.
3. Display Coordination (DCO). This CSCI comprised user interface control, display formats, and display population.
4. Test and Simulation (TAS). This CSCI comprised test scenario generation, test message injection, data recording, and scenario playback.
5. Common Mission Processing (CMP). This CSCI comprised the missile warning algorithms for radar, nuclear detonation, and satellite early warning messages.
6. Common Communications (CCO). This CSCI comprised external interfaces with other systems and message input, output, and protocol management.

Table D-1 summarizes the distinguishing characteristics of each CSCI.





FSD Project Organization and Responsibilities

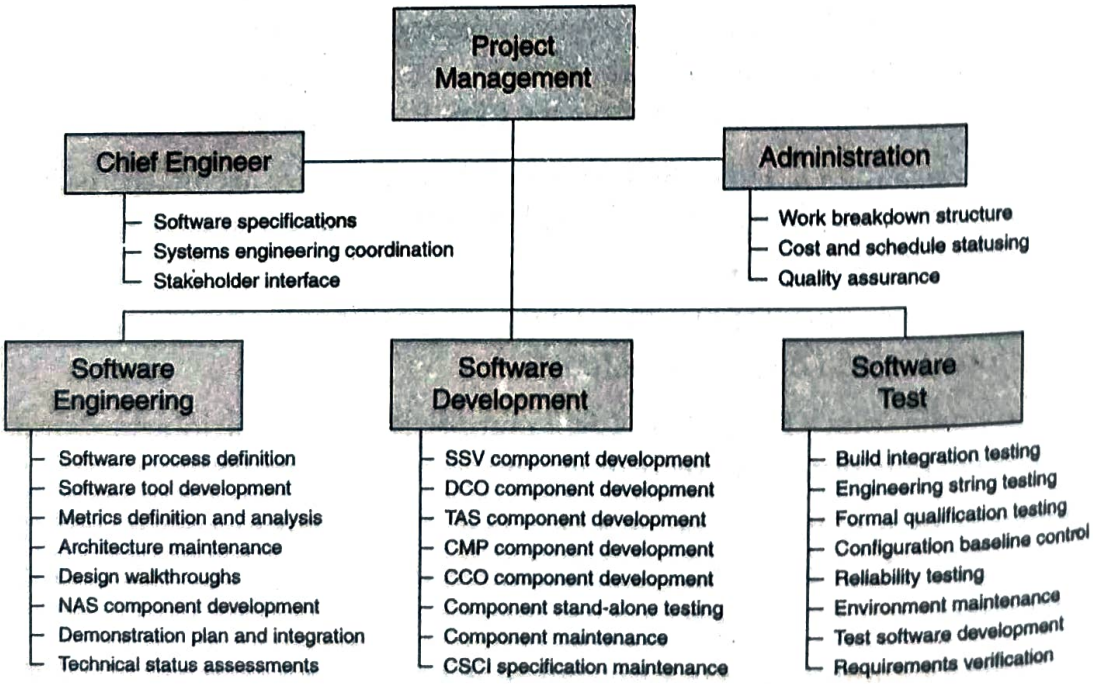


FIGURE D-2. *Full-scale development phase project organization*

TABLE D-1. *CSCI summary*

CSCI	SIZE (SLOC)	COMPLEXITY	ASSETS (+) AND CHALLENGES (–)
NAS	20,000	Very high	+ Experienced, superior team; second-generation product – Reusable across subsystems; high performance, reliability
SSV	160,000	High	+ Some tool-produced code; stable NAS primitives – Numerous global interfaces, types, components
DCO	70,000	Moderate	+ Flexible display design; tool-produced formats – Tough performance requirement; continuous change
TAS	10,000	Low	+ Simple application; some off-line processing – Offsite team; limited environment resources
CMP	15,000	Moderate	+ Domain experience; straightforward processing – Offsite team; stringent performance requirements; numerous stakeholders involved in approving algorithm design
CCO	80,000	High	+ Skilled people; precedent domain experience – Stringent performance requirements; unstable external interface
Total	355,000	High	Large scale, high performance, high reliability

The Software Architecture Skeleton

The CCPDS-R software process was tailored to exploit Ada and reusable middleware components in order to construct a distributed architecture rapidly. The NAS CSCI provided these primitive components and was initially developed on independent research and development funding before the CCPDS-R contract was awarded. These components were a first-generation middleware solution that enabled a true component-based development approach to distributed architectures. The instantiation of the NAS generic tasks, processes, sockets, and circuits into a run-time infrastructure was called a software architecture skeleton (SAS). The software engineering associated with the Common Subsystem SAS was the focus of the early builds and demonstrations. This is an excellent example of an architecture-first process.

The SAS encompasses the declarative view of the solution, including all the top-level control structures, interfaces, and data types passed across these interfaces. In the CCPDS-R definition of an architecture, this view included the following:

- All Ada main programs
- All Ada tasks and task attributes
- All sockets (asynchronous task-to-task communications), socket attributes, and connections to other sockets
- Data types for objects passed across sockets
- NAS components for initialization, state management of processes and tasks, interprocess communications, fault handling, health and performance monitoring, instrumentation, network management, logging, and network control

Even though a SAS will compile, it will not really execute many scenarios (except to come up and idle) unless software is added that reads messages, processes them, and writes them within application tasks. The purpose of a SAS is to provide the structure and interface network for integrating components into threads of capability. There are two important aspects of SAS verification and assessment: compilation and execution. Merely constructing and compiling all the SAS objects together is an important and nontrivial assessment that will provide substantial feedback about the consistency and quality of the SAS. Constructing components and executing stimuli and response threads within the SAS provide further feedback about structural integrity and run-time semantics.

The SAS, then, provides the forum for integration and architecture evolution. It is important to construct the SAS early and to evolve it into a stable baseline in which change is managed and measured for feedback about architecture stability. CCPDS-R installed its first SAS baseline (after three informal iterations) around month 13, just before the preliminary design review (PDR) milestone; all subsequent change was performed via rigorous configuration control. The SAS underwent numerous changes after its first baseline. These changes were scrutinized closely as the project progressed, but the SAS dynamics converged on an acceptable architecture with solid substantiation early in the life cycle. The SAS was useful in assessing the volatility in the overall software interfaces and captured the conceptual architecture of the Common Subsystem.

Figure D-3 provides a perspective of the software architecture stability. The graphs show that there was significant architectural change over the first 20 months of the project, after which the architecture remained pretty stable. The large spike in processes and tasks around month 5 corresponded to an attempt at a much more

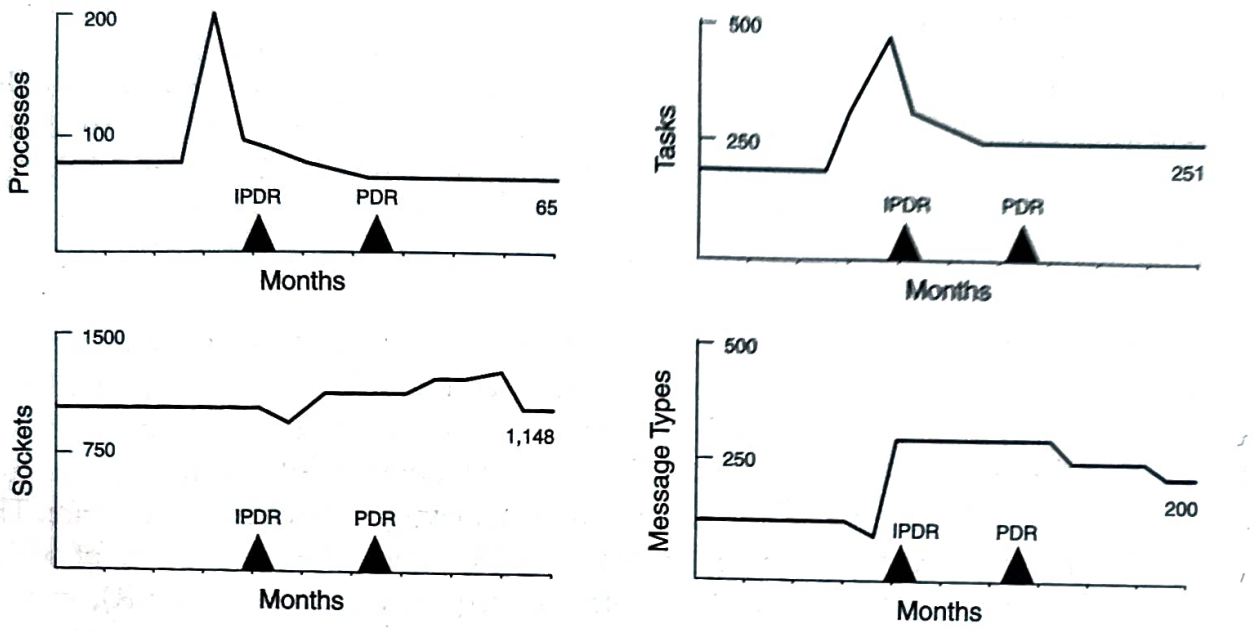


FIGURE D-3. *Common Subsystem SAS evolution*

distributed approach. As this architecture experimentation exposed the design trade-offs in distribution strategies, the SAS process design was changed back to the original number of processes, but the SAS task-level design converged on an increased number of tasks. The basic problems being examined by the architecture team were the trade-offs in concurrency, operating system process overhead, run-time library tasking overhead, paging, context switching, and the mix of interprocess, intertask, and internode message exchange. The complexity of such run-time interactions made modeling and simulation ineffective. Only the early run-time demonstrations of multiple distribution configurations allowed the architecture team to achieve the understanding of technical trade-offs necessary to select an adequate solution. If the change in the distribution design had occurred very late in the project, the impact could have been immense. Because sockets and messages were fairly simple to change and corresponded to lower level application interfaces, changes to these numbers continued at a low level through the critical design review (CDR) milestone.

The freedom to experiment with an architecture proved to be very valuable to the achievement of an adequate architecture baseline early in the life cycle. This was enabled primarily by the flexibility of the NAS CSCI.

CCPDS-R was seriously committed to an architecture-first approach. The "architecture" description of CCPDS-R was centered on the process view, as described in Chapter 7. This was due primarily to the stringent run-time performance requirements and the risks associated with a first-generation distributed architecture.